# GUÍA EXPLICATIVA DE TODO EL CÓDIGO DIVIDIDA POR MÓDULOS

**Proyecto: Sangre y Dogma: Arena Elemental (C# / SharpDevelop 5 / Windows Forms)**

Materia: Algoritmos y Programación I (ALP-265) - II Semestre - IUJO Barquisimeto (Prof. Pablo Lucena). Esta guía desglosa cada archivo (.cs) del proyecto con lenguaje sencillo para que cualquier integrante del grupo pueda defenderlo línea por línea.

---

## MÓDULO 1: INICIO Y PORTADA (Program.cs y PantallaInicioForm.cs)

---

1. Program.cs: Es el punto de entrada (Main). Abre primero PantallaInicioForm con Application.Run(new PantallaInicioForm());

2. PantallaInicioForm.cs: Tiene un PictureBox (picPortada) con Dock = Fill y SizeMode = Zoom, y un panel gris sencillo abajo con los botones 'Iniciar Juego' y 'Salir'.

```
private void CargarImagenPortada()
{
try {
if (File.Exists("portada.jpg")) picPortada.Image = Image.FromFile("portada.jpg");
} catch { }
}
```

¿Cómo defenderlo? Usamos un bloque try/catch sencillo al cargar portada.jpg para que si algún día se mueve la imagen de carpeta, el programa no se cierre ni dé error.

---

## MÓDULO 2: MENÚ PRINCIPAL (MainForm.cs)

---

¿Para qué sirve? Tiene el estilo gris clásico de Windows Forms con un GroupBox ('Menú Principal') y 7 botones que abren cada ventana del proyecto usando ShowDialog().

```
public GestorEstructuras Gestor;
public ConexionBD BaseDatos;
public MainForm()
{
InitializeComponent();
Gestor = new GestorEstructuras();
BaseDatos = new ConexionBD();
}
```

¿Qué hace cada botón en MainForm?

- Botón 1 (BtnModoHistoriaClick): Abre FormCombate pasando true para pelear contra los Duendes.

- Botón 2 (BtnModo2JugadoresClick): Abre FormCombate pasando false para pelear Jugador 1 vs Jugador 2.

- Botón 3 (MOSTRAR HISTORIA - BtnLoreClick): Abre FormLore para consultar la pequeña historia (contexto y motivo) y los atributos de cada personaje.

- Botón 4 (BtnGeneradorMazosClick): Abre FormGeneradorMazos para crear cartas (suma = 100) y armar los mazos de 5 cartas.

- Botón 5 (BtnGestionMazosClick): Abre FormGestionBD para guardar jugadores y estadísticas en SQL.

- Botón 6 (BtnCombateRedClick): Abre FormCombateRed para jugar en red local (LAN).

- Botón 7 (BtnLeerTxtClick): Lee el archivo PuntajesMaximos.txt con StreamReader y lo muestra en un MessageBox.

---

## MÓDULO 3: CLASES, HERENCIA, POLIMORFISMO Y CÁLCULO (Personaje.cs)

---

¿Para qué sirve? Cumple todo el 1er Cohorte (POO). Nota importante: esta clase NO tiene ningún MessageBox ni controles visuales, respetando la separación entre lógica y pantalla.

### 1. Encapsulamiento con get y set en la clase padre Personaje:

```
private int hp;
private int def;
private int atk;
public int HP  { get { return hp; }  set { hp = value; } }
public int DEF { get { return def; } set { def = value; } }
public int ATK { get { return atk; } set { atk = value; } }
```

Explicación: Los atributos (id, nombre, elemento, hp, def, atk, ener, esp, emp) son privados (private) y se leen o modifican con propiedades públicas get y set. En todos los personajes se cumple que HP + DEF + ATK = 100, empiezan con ENER = 2 y ESP = 0.

### 2. Método virtual Atacar() — Fórmula de Cálculo y Triángulo Elemental:

```
public virtual int Atacar(Personaje enemigo)
{
double danoCalculo = (double)(ATK * ATK) / (double)(ATK + enemigo.DEF);
if (Elemento == "Viento" && enemigo.Elemento == "Fuego") danoCalculo = danoCalculo * 1.5;
else if (Elemento == "Fuego" && enemigo.Elemento == "Tierra") danoCalculo = danoCalculo * 1.5;
else if (Elemento == "Tierra" && enemigo.Elemento == "Agua") danoCalculo = danoCalculo * 1.5;
else if (Elemento == "Agua" && enemigo.Elemento == "Viento") danoCalculo = danoCalculo * 1.5;
int danoFinal = (int)Math.Round(danoCalculo);
if (danoFinal < 1) danoFinal = 1;
enemigo.HP = enemigo.HP - danoFinal;
return danoFinal;
}
```

¿Cómo explicar la fórmula al profesor? Si restáramos (ATK - DEF), cuando la defensa es mayor que el ataque daría un daño negativo. Con la división (ATK * ATK) / (ATK + DEF) aplicamos un límite matemático: aunque la defensa rival sea muy grande, el daño se achica acercándose a 0 pero nunca da negativo. Luego, con if/else revisamos el triángulo elemental (Viento > Fuego > Tierra > Agua > Viento) y si hay ventaja multiplicamos por 1.5.

### 3. Herencia y Polimorfismo (Protagonista, Duende y CartaEscenario):

- Protagonista : Personaje: Hereda de Personaje y agrega el atributo Rol (Guerrero, Maga, Sanador, Alquimista, Brujo). Usa override en EjecutarHabilidadEspecial() para que cada rol haga su propio efecto (ej. golpe fijo, llamarada doble, curación, ácido o robo de energía).

- Duende : Personaje: Hereda de Personaje y agrega Nivel (1 al 7). Usa override en EjecutarHabilidadEspecial() para pegar según su nivel: 10 + (Nivel * 3).

- CartaEscenario: Tiene AplicarBuffNerf() (+4 ATK al mismo elemento, -2 DEF al resto) y DispararTerremoto(), que revisa si la Empatía (EMP) es mayor o igual a 50 para curar (+10 HP, +4 DEF) o quitar vida (-8 HP).

---

## MÓDULO 4: ARREGLOS, LISTAS, COLAS, PILAS Y ARCHIVOS TXT (GestorEstructuras.cs)

---

¿Para qué sirve? Cumple el 2do y 3er Cohorte reuniendo todas las estructuras de datos y el manejo de archivos .TXT.

### 1. Arreglo y Listas dinámicas (List<T>):

```
public static Personaje[] Catalogo12Personajes = new Personaje[12];
public List<Personaje> ListaPersonajes;
public List<Personaje> MazoGuardadoJ1;
public List<Personaje> MazoGuardadoJ2;
```

- Guardamos los 12 personajes base (3 de Agua, 3 de Tierra, 3 de Viento y 3 de Fuego) en el arreglo Catalogo12Personajes y los pasamos con un ciclo for y .Add() a ListaPersonajes.

- El método QuitarPersonajeDeMazo() usa explícitamente mazo.Remove(elegido) para demostrar el borrado en listas.

### 2. Colas (Queue) y Pila (Stack):

```
public Queue<Personaje> ColaJugador1;
public Queue<Personaje> ColaJugador2;
public Stack<string> PilaHistorial;
```

- En PrepararColasCombate() recorremos las listas con foreach y metemos los 5 personajes en las Colas con .Enqueue() (FIFO: el primero en entrar es el primero en salir a pelear con .Dequeue()).

- PilaHistorial usa .Push() para apilar cada turno del combate (LIFO).

### 3. Archivos de Texto (.TXT) con StreamWriter y StreamReader:

- GuardarTxt() escribe todas las líneas de la partida en HistorialBatalla.txt.

- RegistrarPuntajeMaximoTxt() y LeerPuntajesMaximosTxt() escriben y leen el archivo PuntajesMaximos.txt.

---

## MÓDULO 5: SIMULADOR DE COMBATE POR TURNOS (FormCombate.cs)

---

¿Para qué sirve? Controla toda la partida local (1 Jugador vs Duendes o 2 Jugadores).

### Puntos clave que pide el Enunciado y cómo están hechos:

1. Sorteo inicial aleatorio: Al abrir la ventana usamos Random aleatorio = new Random(); Turno = aleatorio.Next(1, 3); para decidir al azar quién empieza.

2. Economía de Energía (ENER): Cada personaje empieza con 2 de energía. Atacar cuesta 1 ENER. Si al jugador todavía le queda energía (atacante.ENER > 0), puede volver a actuar en su mismo turno o darle a 'Pasar Turno' para ahorrar energía. Al cambiar de turno se suma +1 ENER.

3. Habilidad Especial (ESP): Se muestra en una ProgressBar (de 0 a 2). Está bloqueada en la Ronda 1 y tarda 2 turnos en llenarse al 100%. Cuando tiene 2 ESP y 2 ENER, el botón se habilita.

4. Relevos con Cola (BtnRelevoClick): Mete al personaje actual al final de la fila con ColaJugador1.Enqueue(CartaJ1) y saca al siguiente con CartaJ1 = ColaJugador1.Dequeue().

5. Reloj de turno (TimerTurnoTick) y botón MOSTRAR HISTORIA: Da 15 segundos por turno y permite pulsar 'MOSTRAR HISTORIA' para leer el contexto y motivo de los dos personajes que están peleando.

---

## MÓDULO 6: GENERADOR DE MAZOS Y MOSTRAR HISTORIA (FormGeneradorMazos.cs y FormLore.cs)

---

1. FormGeneradorMazos.cs: Permite crear cartas nuevas validando con int.TryParse que sean números enteros y comprobando con un if que HP + DEF + ATK sumen exactamente 100:

```
int suma = hp + def + atk;
if (suma != 100) { MessageBox.Show("HP + DEF + ATK debe sumar 100"); return; }
```

También permite a ambos jugadores agregar 5 personajes a su mazo con .Add() o quitar uno seleccionado con .Remove().

2. FormLore.cs (MOSTRAR HISTORIA): Muestra la pequeña historia (contexto y motivo) guardada en la propiedad p.Historia de cada uno de los 12 personajes y 4 escenarios, junto con su ficha técnica y el botón 'MOSTRAR HISTORIA'.

---

## MÓDULO 7: BASE DE DATOS SQL Y RED LAN (ConexionBD.cs, FormGestionBD.cs y FormCombateRed.cs)

---

### 1. Base de Datos SQL con Parámetros (ConexionBD.cs y FormGestionBD.cs):

Conecta con SQL Server en 3 tablas simples relacionadas por Clave Primaria (PK: IdJugador) y Clave Foránea (FK): Jugadores, Personajes y Estadisticas. Todas las consultas usan SqlParameter para no concatenar texto:

```
string sqlJugador = "INSERT INTO Jugadores (Nombre) OUTPUT INSERTED.IdJugador VALUES
(@Nombre)";
SqlCommand cmd1 = new SqlCommand(sqlJugador, conexion);
cmd1.Parameters.Add(new SqlParameter("@Nombre", nombreJugador));
```

En FormGestionBD.cs se calculan las Partidas Jugadas (Victorias + Derrotas) y el Porcentaje de Éxito ((Victorias / Partidas) * 100). Si la PC no tiene SQL Server prendido, el catch guarda todo en RespaldoBD.txt para que nunca falle.

### 2. Combate en Red LAN (FormCombateRed.cs):

Usa TcpListener (Servidor - Jugador 1) y TcpClient (Cliente - Jugador 2) en el puerto 5000 enviando mensajes de texto sencillos separados por comas (ejemplo: 'ATAQUE,15,Fabian ataca') que se separan con paquete.Split(',').